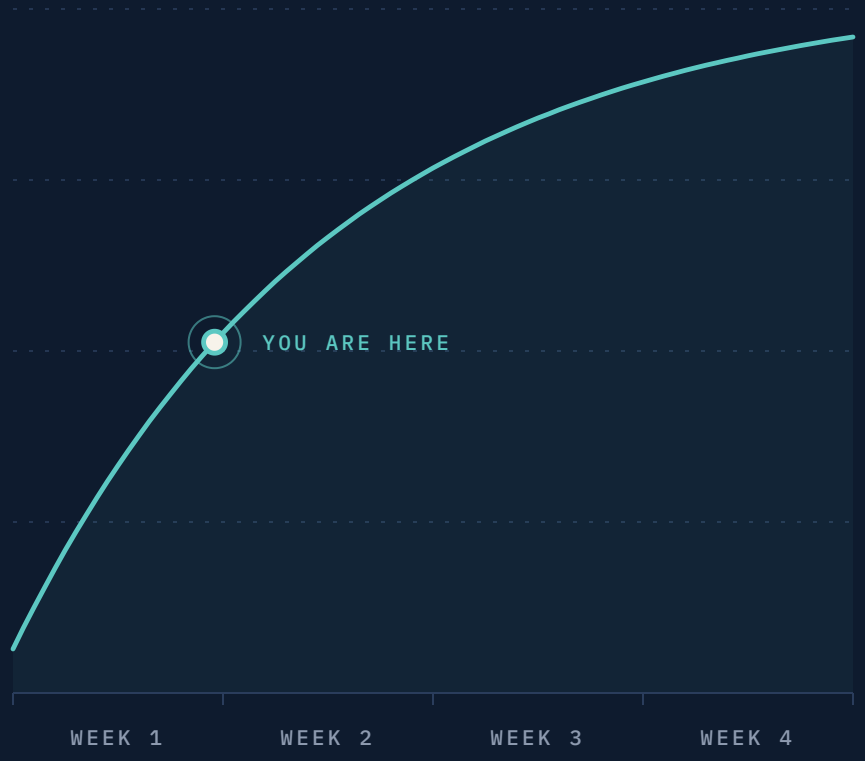
WEEK 1 · DAY 5 · OVERVIEW

Case Study

Organizing a Small Portfolio



Created by **Jeff Lenamon**

ABOUT 50 MIN WITH EXERCISES / 7 SECTIONS / 6 EXERCISES

This course is educational content created in a personal capacity. Nothing here is investment advice or a recommendation to buy or sell any security. All portfolio data is synthetic.

Problem Definition

Another team has handed us OSTV and VYRH. Their email lists all six of our bonds, typed into the body: no file, no system record. Put the book in order before anyone analyzes it, make sure it ties to the email's 4,400,000 face, and hand it on as a file.

ABOUT 50 MIN WITH EXERCISES / 7 SECTIONS / 6 EXERCISES

Your Route Through Day 5

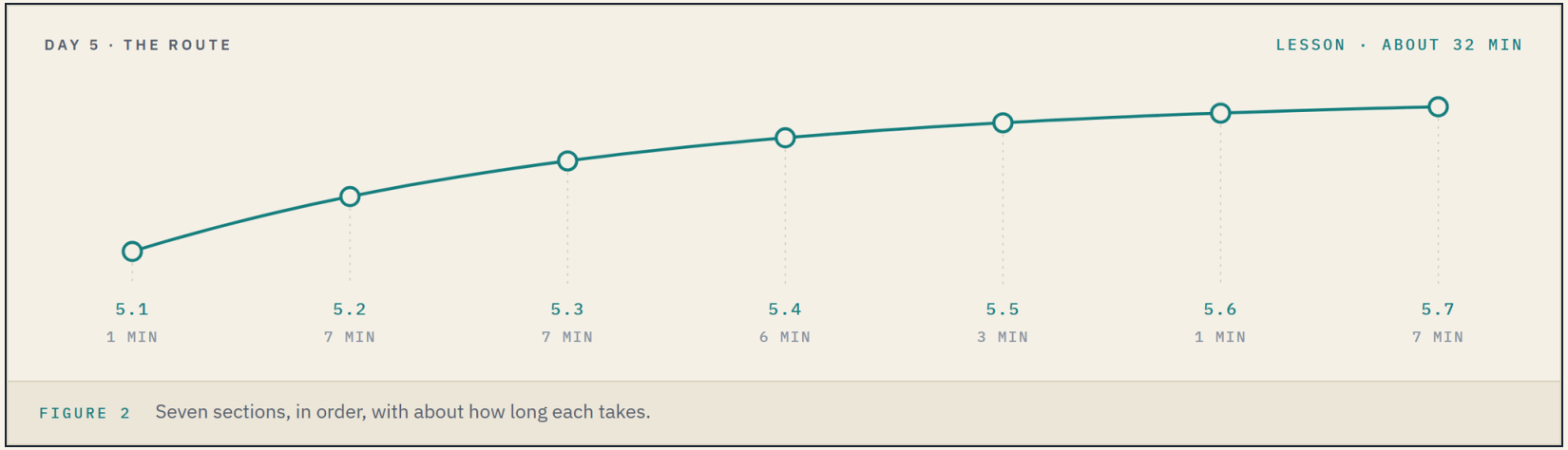


FIGURE 2 Seven sections, in order, with about how long each takes.

	SECTION	ABOUT
5.1	Storing Bonds as Variables	1 min
5.2	Organizing and Querying the Data	7 min
5.3	Updating Data Based on Conditions	7 min
5.4	A Modular Way of Fetching Data	6 min

	SECTION	ABOUT
5.5	Combining All the Data Into a Single Element	3 min
5.6	Converting the Dictionary Into a pandas DataFrame	1 min
5.7	Accessing and Analyzing the Data	7 min

Time. About 32 minutes for the lesson, then about 18 for the six exercises.

5.1 Storing Bonds as Variables

THE CODE

```
ticker1 = 'QVNE'
ticker2 = 'DNMR'
ticker3 = 'PLWK'
ticker4 = 'HLVF'
ticker5 = 'OSTV'
ticker6 = 'VYRH'
print(ticker1, ticker2, ticker3, ticker4, ticker5, ticker6)
```

```
QVNE DNMR PLWK HLVF OSTV VYRH
```

One variable per value does not scale. Store each column as a list.

5.2 Organizing and Querying the Data

TIE THE LISTS TO THE EMAIL

```
total_face_before = sum(face_list)
assert total_face_before == 4400000, 'the face does not tie to the email'
print(f'{len(ticker_list)} bonds with a total face of {total_face_before:,}')
```

```
6 bonds with a total face of 4,400,000
```

PRINT EVERY LIST

```
print('Tickers:', ticker_list)
print('Face held:', face_list)
```

```
Tickers: ['QVNE', 'DNMR', 'PLWK', 'HLVF', 'OSTV', 'VYRH']
Face held: [1500000, 500000, 750000, 250000, 1000000, 400000]
```

EXCEL SIDE BY SIDE

Excel side by side. The check cell `=IF(SUM(H2:H7)=4400000,"OK","CHECK")` shows OK, and `=COUNTA(A2:A7)=COUNT(H2:H7)` shows TRUE.

A check cell calculates on; an **`assert`** stops the run.

The lists tie to the email. Now fetch one bond.

Find a Bond, Then Read Its Row

Read down one column: DNMR's whole record

index	0	1	2	3	4	5
ticker_list	QVNE	**DNMR**	PLWK	HLVF	OSTV	VYRH
sector_list	Energy	**Transportation**	Materials	Consumer	Telecom	Health Care
score_list	9	**11**	7	14	10	12
coupon_list	5.25	**6.125**	4.75	7.5	5.75	6.75
price_list	98.5	**103.25**	95.0	101.0	99.0	102.5
face_list	1500000	**500000**	750000	250000	1000000	400000
MATCH	1	**2**	3	4	5	6

Python counts from 0, MATCH from 1: the same bond

FIGURE 3 The six lists as rows: DNMR's record sits at index 1 in every list, where Excel's MATCH counts it as 2.

FIND THE POSITION

```
pos = ticker_list.index('DNMR')
print(pos)
```

```
1
```

READ THE ROW

```
print(ticker_list[pos])
print(sector_list[pos])
print(score_list[pos])
```

```
DNMR
Transportation
11
```

A full record for any bond. Then news arrives.

A Downgrade

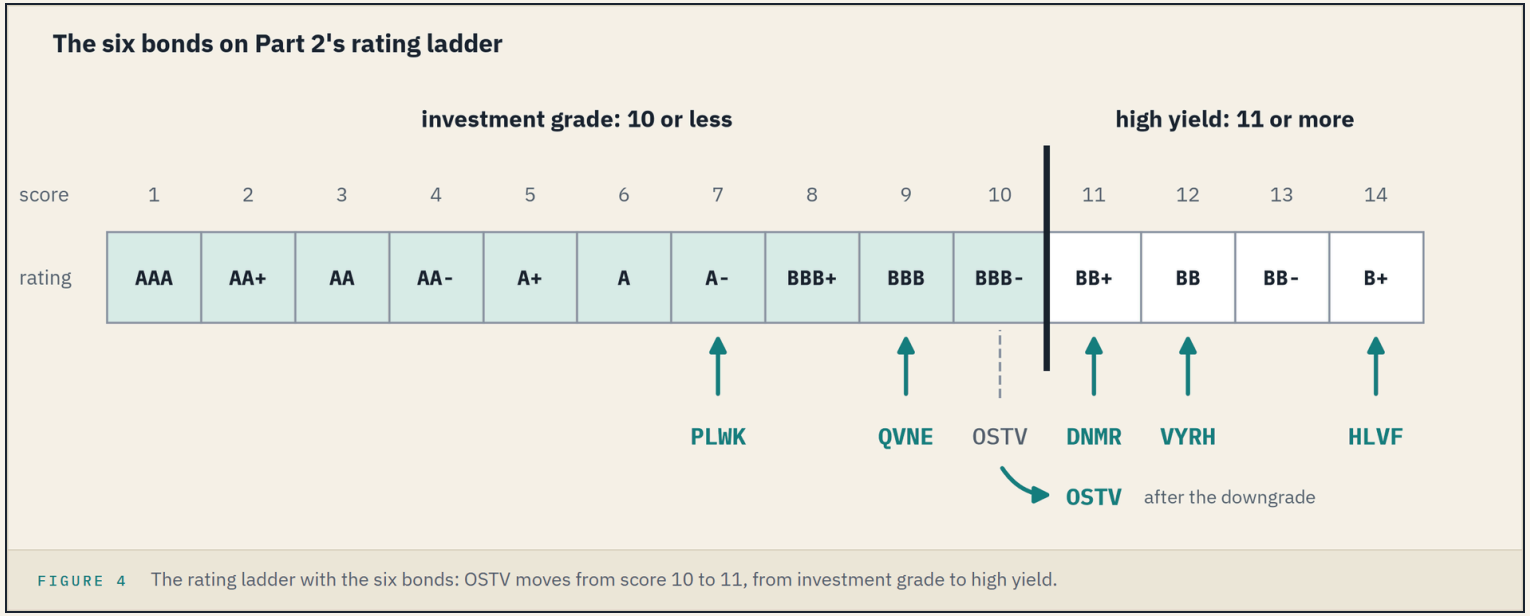


FIGURE 4 The rating ladder with the six bonds: OSTV moves from score 10 to 11, from investment grade to high yield.

UPDATE THE RECORD

```
pos = ticker_list.index('OSTV')
score_list[pos] = 11
print(score_list)
```

```
[9, 11, 7, 14, 11, 12]
```

OSTV has crossed from investment grade to high yield.

5.3 Updating Data Based on Conditions

ONE LOOP FOR EVERY BOND

```
price_list = email_price_list.copy()
for i, score in enumerate(score_list):
    if score >= 11:
        price_list[i] = price_list[i] - 0.5
        print('Corrected', ticker_list[i])
print(price_list)
```

```
Corrected DNMR
Corrected HLVF
Corrected OSTV
Corrected VYRH
[98.5, 102.75, 95.0, 100.5, 98.5, 102.0]
```

COUNT BY A CONDITION

```
investment_grade = 0
high_yield = 0
for score in score_list:
    if score <= 10:
        investment_grade += 1
    else:
        high_yield += 1
print('Investment grade:', investment_grade)
print('High yield:', high_yield)
```

```
Investment grade: 2
High yield: 4
```

The same lines would correct 200 bonds. Now ask the book questions.

5.4 A Modular Way of Fetching Data

THE FUNCTION

```
def bond_info(ticker):
    """Print everything the desk holds on one bond."""
    pos = ticker_list.index(ticker)
    print(f'Ticker: {ticker_list[pos]}')
    print(f'Sector: {sector_list[pos]}')
    print(f'Score: {score_list[pos]}')
    print(f'Coupon: {coupon_list[pos]:.3f} percent')
    print(f'Price: {price_list[pos]:.2f}')
    print(f'Face held: {face_list[pos]:,}')
    print('-' * 30)
```

LOOK UP TWO BONDS

```
bond_info('QVNE')
```

```
Ticker: QVNE
Sector: Energy
Score: 9
Coupon: 5.250 percent
Price: 98.50
Face held: 1,500,000
------------------------------
```

Any bond the desk holds, in one line. Now one it does not hold.

5.5 Combining All the Data Into a Single Element

ONE DICTIONARY

```
portfolio = {
    'Ticker': ticker_list,
    'Sector': sector_list,
    'Score': score_list,
    'Coupon': coupon_list,
    'Price': price_list,
    'Face': face_list,
}
portfolio
```

```
{'Ticker': ['QVNE', 'DNMR', 'PLWK', 'HLVF', 'OSTV', 'VYRH'],
 'Score': [9, 11, 7, 14, 11, 12],
 'Coupon': [5.25, 6.125, 4.75, 7.5, 5.75, 6.75],
 'Price': [98.5, 102.75, 95.0, 100.5, 98.5, 102.0],
 'Face': [1500000, 500000, 750000, 250000, 1000000, 400000]}
```

ONE ITEM FROM A COLUMN

```
print(portfolio['Ticker'][2])
```

```
PLWK
```

EVERY COLUMN AT ONE POSITION

```
for key, value in portfolio.items():
    print(key, value[3])
```

```
Ticker HLVF
Sector Consumer
Score 14
Coupon 7.5
Price 100.5
Face 250000
```

One name now holds the book.

5.6 Converting the Dictionary Into a pandas DataFrame

THE CODE

```
import pandas as pd
df = pd.DataFrame(portfolio)
df
```

THE BOOK AS A TABLE

	TICKER	SECTOR	SCORE	COUPON	PRICE	FACE
0	QVNE	Energy	9	5.250	98.50	1500000
1	DNMR	Transportation	11	6.125	102.75	500000
2	PLWK	Materials	7	4.750	95.00	750000
3	HLVF	Consumer	14	7.500	100.50	250000
4	OSTV	Telecom	11	5.750	98.50	1000000
5	VYRH	Health Care	12	6.750	102.00	400000

The email is now a table. Ask it questions.

5.7 Accessing and Analyzing the Data

EVERY COLUMN IN ONE LINE

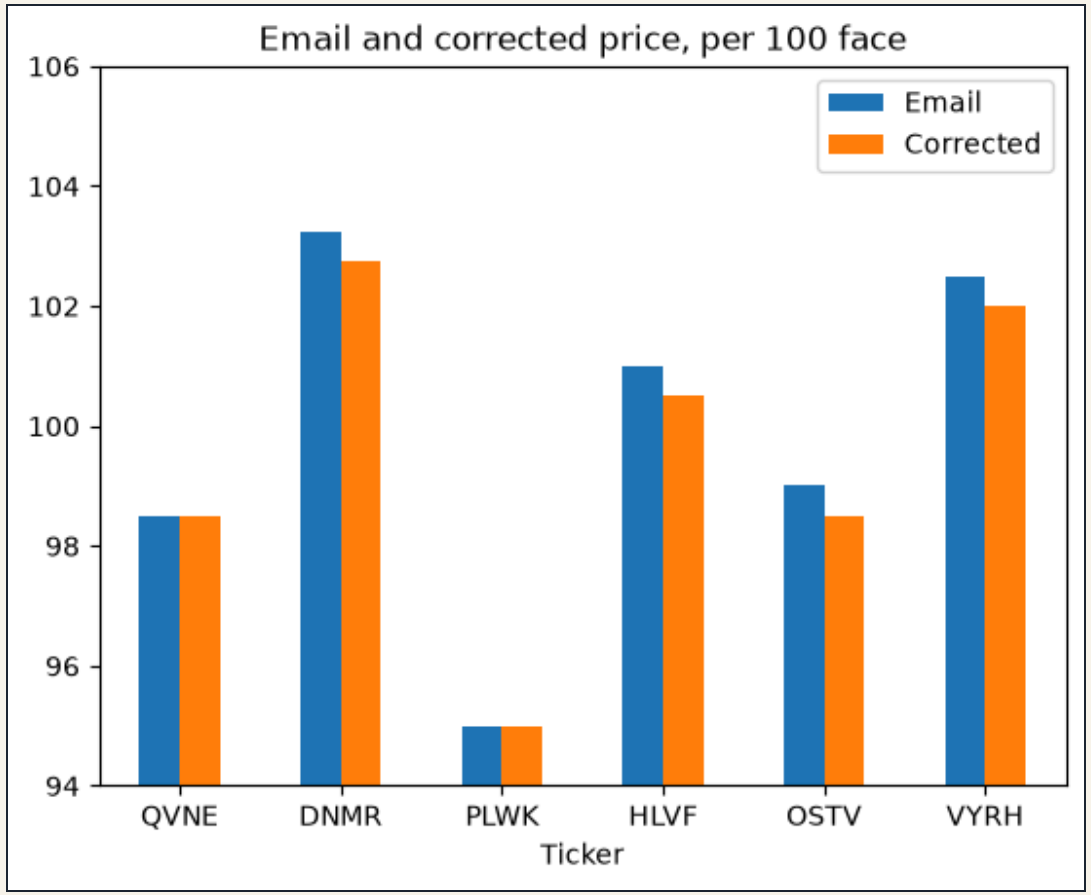
	SCORE	COUPON
count	6.000000	6.000000
mean	10.666667	6.020833
std	2.422120	1.001301
min	7.000000	4.750000
25%	9.500000	5.375000
50%	11.000000	5.937500
75%	11.750000	6.593750
max	14.000000	7.500000

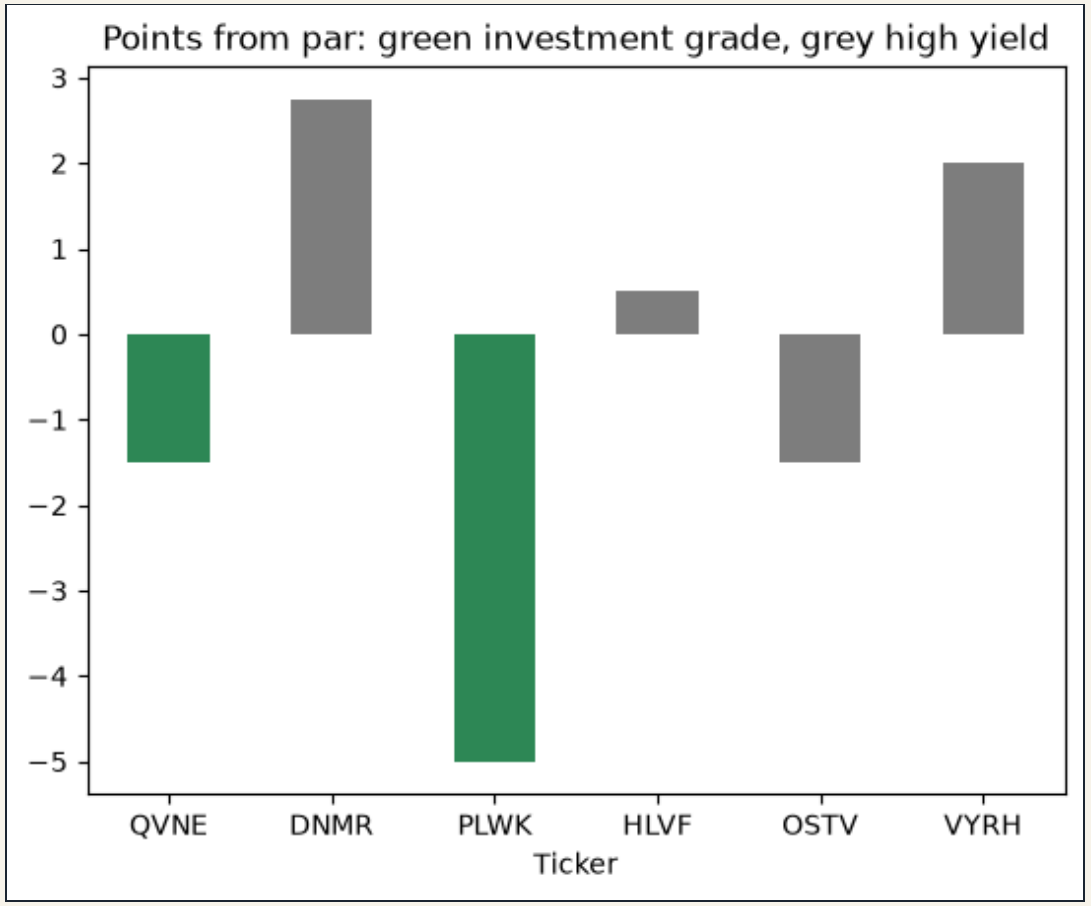
EXCEL SIDE BY SIDE

ROW	MEANS	EXCEL
count	how many values	`=COUNT(I2:I7)`
mean	the average	`=AVERAGE(I2:I7)`
std	standard deviation: how spread out the values are	`=STDEV.S(I2:I7)`
min, max	the two ends	`=MIN(I2:I7)`, `=MAX(I2:I7)`
25%, 75%	a quarter, and three quarters, of the way up	`=QUARTILE.INC(I2:I7,1)`, `=QUARTILE.INC(I2:I7,3)`
50%	the median, the middle value	`=MEDIAN(I2:I7)`

The numbers summarize; a chart shows each bond.

What the Correction Did





Three above par, three below; four of six high yield.

You Can Now

- Look up and update any bond by its position in a list.
- Apply a change to the right bonds with a condition inside a loop.
- Count what you need, the way **`COUNTIF`** does.
- Turn a lookup into a function, and return the answer.
- Gather lists into a dictionary, turn it into a DataFrame, and save it as a CSV file.
- Tie a total to its source with **`assert`**, before and after a change.

NEXT
WEEK 2 · DAY 1, PART 1

NumPy, Part 1

NumPy does the arithmetic on a whole column of bonds in a single line.

